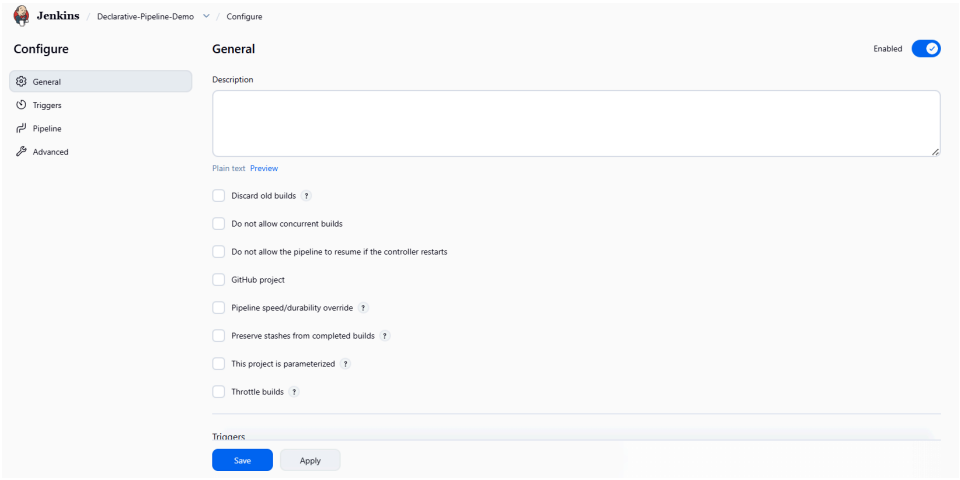
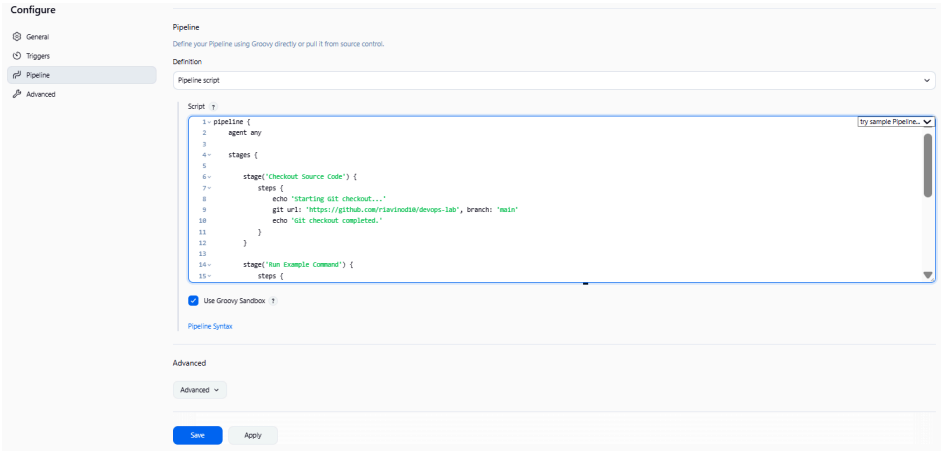
SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Symbiosis International (Deemed University)
(Established under section 3 of the UGC Act, 1956)
Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC
Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 06

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	**Deepak Rathod**
PRN No.	**23070122175**
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	16th July, 2026
Title of Assignment:	Basic Declarative Pipeline
Theory:	**Theory** A Jenkins Pipeline is a suite of plugins that enables Continuous Integration and Continuous Delivery (CI/CD) by defining the entire software development workflow as code. It allows developers to automate tasks such as source code checkout, building, testing, and deployment. A Declarative Pipeline provides a structured and simplified syntax for defining pipeline stages using Groovy. It consists of key components such as pipeline, agent, stages, stage, and steps. The agent specifies where the pipeline will run, stages divide the workflow into logical phases, and steps define the actions to be executed within each stage. In this experiment, a Declarative Pipeline project was created in Jenkins. The pipeline checked out source code from a GitHub repository, executed shell commands to verify the repository contents, and displayed the build output. The successful execution of each stage was verified through the Stage View and Console Output. **Commands Used** **Pipeline Script Components** • pipeline {} – Defines the complete Declarative Pipeline. • agent any – Specifies that the pipeline can run on any available

	Jenkins agent. • stages {} – Groups all stages of the pipeline. • stage('Checkout Source Code') – Defines the stage for retrieving source code from GitHub. • git url: 'https://github.com/riavinod10/devops-lab', branch: 'main' – Checks out the source code from the specified GitHub repository. • stage('Run Example Command') – Defines the stage for executing shell commands. • echo 'message' – Displays informational messages in the Jenkins build log. • sh 'echo "Hello from Declarative Pipeline!"' – Executes a shell command to print a sample message. • sh 'cat README.md' – Displays the contents of the README.md file from the repository. • post { always { ... } } – Executes the specified actions after the pipeline finishes, regardless of the build result.
Implementation:	**Step 1: Create a New Pipeline Project**  Created Pipeline Project — Created a new Jenkins Pipeline project named Declarative-Pipeline-Demo by selecting the Pipeline project type. **Step 2: Define the Pipeline Script**



Script:

```
pipeline {
    agent any

    stages {

        stage('Checkout Source Code') {
            steps {
                echo 'Starting Git checkout...'
                git url: 'https://github.com/riavinod10/devops-lab', branch: 'main'
                echo 'Git checkout completed.'
            }
        }

        stage('Run Example Command') {
            steps {
                echo 'Executing a shell command...'

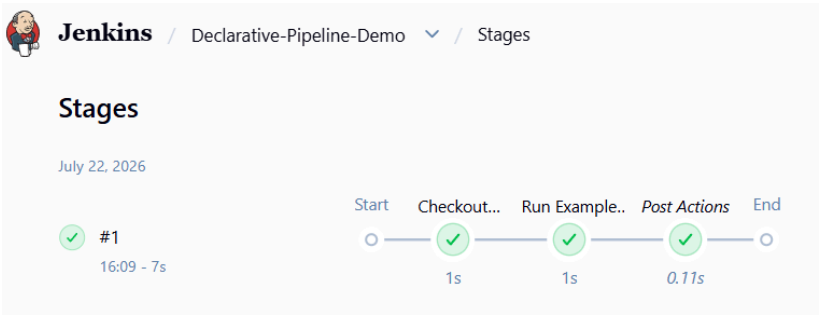
                sh 'echo "Hello from Declarative Pipeline!"'
                sh 'echo "--- Reading README ---"'
                sh 'cat README.md'
            }
        }

    }

    post {
        always {
            echo 'Pipeline finished.'
        }
    }
}
```

Defined Declarative Pipeline Script — Configured the Jenkins Pipeline using a Declarative Pipeline script to check out source code from the GitHub repository and execute shell commands in separate stages.

Step 3: Save and Run the Pipeline



Executed Declarative Pipeline — Successfully executed the Declarative Pipeline and verified that all stages, including Checkout Source Code and Run Example Command, completed successfully.

Step 4: View Console Output

```
Started by user unknown or anonymous
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/Declarative-Pipeline-Demo
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout Source Code)
[Pipeline] echo
Starting Git checkout...
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/riavinod10/devops-lab
 > git init /var/jenkins_home/workspace/Declarative-Pipeline-Demo # timeout=10
Fetching upstream changes from https://github.com/riavinod10/devops-lab
 > git --version # timeout=10
 > git --version # 'git version 2.47.3'
 > git fetch --tags --force --progress -- https://github.com/riavinod10/devops-lab +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git config remote.origin.url https://github.com/riavinod10/devops-lab # timeout=10
 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 245b30d7eaabf375dd382f347da20725bfe4bda0 (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 245b30d7eaabf375dd382f347da20725bfe4bda0 # timeout=10
 > git branch -a -v --no-abbrev # timeout=10
 > git checkout -b main 245b30d7eaabf375dd382f347da20725bfe4bda0 # timeout=10
Commit message: "Merge conflicting-feature, resolve README conflict"
First time build. Skipping changelog.
[Pipeline] echo
Git checkout completed.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Run Example Command)
[Pipeline] echo
Executing a shell command...
[Pipeline] sh
+ echo Hello from Declarative Pipeline!
Hello from Declarative Pipeline!
[Pipeline] sh
```

	+ cat README.md ##### My Awesome DevOps Lab (Resolved Conflict) This update adds details about a new feature. [Pipeline] } [Pipeline] // stage [Pipeline] stage [Pipeline] { (Declarative: Post Actions) [Pipeline] echo Pipeline finished. [Pipeline] } [Pipeline] // stage [Pipeline] } [Pipeline] // node [Pipeline] End of Pipeline Finished: SUCCESS Verified Console Output — Examined the console output and verified that Jenkins successfully checked out the Git repository, executed the pipeline stages, ran the shell commands, and completed the pipeline with a SUCCESS status.

Subject Faculty Name: Prof. Arijit Dutta